

APOSTILA COMPLETA DE

Tkinter & CustomTkinter

Do zero às interfaces modernas em Python

Interfaces gráficas · Widgets · Layouts · Projetos práticos

Python 3.x	Tkinter + CustomTkinter	Projetos incluídos
<i>Compatível com as versões mais recentes</i>	<i>Biblioteca padrão + visual moderno</i>	<i>Calculadora, editor e mais</i>

Sumário

Capítulo 1 — Introdução a GUIs e Tkinter

- › O que é Tkinter?
- › Instalação e configuração
- › Janela principal (Tk)

Capítulo 2 — Widgets Fundamentais

- › Label, Button, Entry
- › Text, Checkbutton, Radiobutton
- › Listbox, Spinbox, Scale
- › Combobox e OptionMenu

Capítulo 3 — Gerenciadores de Layout

- › pack()
- › grid()
- › place()
- › Comparação e boas práticas

Capítulo 4 — Eventos e Callbacks

- › Binding de eventos
- › Variáveis de controle (StringVar, IntVar...)
- › Lambda em callbacks
- › Eventos de teclado e mouse

Capítulo 5 — Frames, Canvas e Menus

- › Frame e LabelFrame
- › Canvas: desenho e animações
- › Menu e Menubutton
- › Toplevel (janelas secundárias)

Capítulo 6 — Diálogos e Messagebox

- › messagebox
- › filedialog
- › simpledialog
- › colorchooser

Capítulo 7 — Estilização com ttk

- › Widgets ttk
- › Style e temas
- › Treeview e Notebook

Capítulo 8 — Introdução ao CustomTkinter

- › O que é CustomTkinter?
- › Instalação
- › Principais diferenças do Tkinter

Capítulo 9 — Widgets do CustomTkinter

- › CtkButton, CtkLabel, CtkEntry
- › CtkCheckBox, CtkRadioButton
- › CtkSlider, CtkProgressBar
- › CtkSwitch, CtkComboBox
- › CtkTextbox

Capítulo 10 — Layouts e Temas no Ctk

- › CtkFrame e CtkScrollableFrame
- › Modos claro/escuro
- › Temas de cores

Capítulo 11 — Projeto 1 — Calculadora (Tkinter)

- › Planejamento da interface
- › Código completo

Capítulo 12 — Projeto 2 — App Moderno (CustomTkinter)

- › Sidebar + área de conteúdo
- › Código completo

Capítulo 13 — Boas Práticas e Próximos Passos

- › Organização em classes
- › Threading com GUI
- › Empacotamento com PyInstaller
- › Recursos para continuar

Introdução a GUIs e Tkinter

Uma **GUI** (Graphical User Interface) permite que usuários interajam com programas através de janelas, botões, menus e outros elementos visuais, em vez de digitar comandos num terminal. Python oferece diversas bibliotecas para criar GUIs, sendo **Tkinter** a mais popular por vir incluída na instalação padrão do Python.

1.1 O que é Tkinter?

Tkinter é a interface Python para o toolkit gráfico **Tk**, desenvolvido originalmente para a linguagem Tcl. Por ser **multiplataforma** (Windows, macOS e Linux), portátil e já embutido no Python, é a primeira escolha para quem quer criar janelas rapidamente sem instalar nada extra.

- Já vem com o Python — sem instalação adicional
- Funciona em Windows, macOS e Linux
- Ótimo para aprender os conceitos de GUI
- Amplamente documentado e com grande comunidade

■ Quando usar Tkinter?

Para ferramentas internas, scripts com interface, protótipos rápidos e projetos acadêmicos, Tkinter é uma excelente escolha. Para apps comerciais com visual moderno, considere CustomTkinter (capítulo 8).

1.2 Instalação e configuração

Verifique se o Tkinter já está disponível no seu Python:

```
# No terminal:

python -m tkinter

# Deve abrir uma pequena janela de demonstração.

# Se aparecer erro, instale via sistema operacional:

# Ubuntu/Debian:

sudo apt-get install python3-tk
```

```
# macOS (Homebrew):  
  
brew install python-tk
```

1.3 Janela Principal — Tk()

Todo programa Tkinter começa criando uma instância de **Tk()**, que representa a janela principal. Depois de adicionar widgets, chamamos **mainloop()** para iniciar o loop de eventos — é ele que mantém a janela aberta e responde às ações do usuário.

```
import tkinter as tk  
  
# Cria a janela principal  
  
janela = tk.Tk()  
  
janela.title('Minha Primeira Janela')  
  
janela.geometry('400x300') # largura x altura em pixels  
  
janela.resizable(True, True) # permite redimensionar  
  
# Inicia o loop de eventos (SEMPRE por último)  
  
janela.mainloop()
```

■ **mainloop() bloqueia!**

Tudo que vem depois de `mainloop()` só será executado quando a janela for fechada. Por isso, configure todos os widgets **ANTES** de chamar `mainloop()`.

Widgets Fundamentais

Widgets são os elementos visuais de uma interface: botões, caixas de texto, rótulos, etc. Todos os widgets do Tkinter seguem o mesmo padrão: você os cria passando o widget pai como primeiro argumento, configura com parâmetros nomeados e depois os posiciona com `pack()`, `grid()` ou `place()`.

2.1 Label — Rótulos

Exibe texto ou imagem estático na tela. Muito usado para títulos e descrições.

```
import tkinter as tk

janela = tk.Tk()

label = tk.Label(janela,

text='Olá, Tkinter!',

font=('Arial', 16, 'bold'),

fg='#1a73e8', # cor do texto

bg='white', # cor de fundo

padx=10, pady=5

)

label.pack()

janela.mainloop()
```

2.2 Button — Botões

Executa uma função (callback) quando clicado.

```
def ao_clicar():

print('Botão clicado!')

label.config(text='Você clicou!')
```

```
btn = tk.Button(janela,  
  
text='Clique aqui',  
  
command=ao_clicar,  
  
bg='#1a73e8',  
  
fg='white',  
  
font=('Arial', 12),  
  
relief='flat',  
  
padx=12, pady=6  
  
)  
  
btn.pack(pady=10)
```

2.3 Entry — Campo de texto de uma linha

Permite ao usuário digitar uma única linha de texto.

```
entry = tk.Entry(janela,  
  
font=('Arial', 12),  
  
width=30,  
  
bd=2,  
  
relief='groove'  
  
)  
  
entry.pack(padx=20, pady=5)  
  
# Ler o valor:  
  
valor = entry.get()  
  
# Inserir valor inicial:
```

```
entry.insert(0, 'Digite aqui...')

# Limpar:

entry.delete(0, tk.END)

# Ocultar senha:

entry_senha = tk.Entry(janela, show='*')
```

2.4 Text — Campo de texto multilinha

```
texto = tk.Text(janela,

height=8, # linhas

width=40, # caracteres

font=('Courier', 11),

wrap='word' # quebra por palavra

)

texto.pack(padx=10, pady=5)

# Inserir conteúdo:

texto.insert(tk.END, 'Linha 1\nLinha 2\n')

# Ler conteúdo:

conteudo = texto.get('1.0', tk.END) # '1.0' = linha 1, col 0

# Limpar:

texto.delete('1.0', tk.END)
```

2.5 Checkbutton e Radiobutton

```

# Checkbutton (múltipla escolha)

var_check = tk.BooleanVar()

check = tk.Checkbutton(janela,

text='Aceito os termos',

variable=var_check,

onvalue=True, offvalue=False

)

check.pack()


# Para ler: var_check.get() → True ou False


# Radiobutton (escolha única)

opcao = tk.StringVar(value='A')

for texto, val in [('Opção A', 'A'), ('Opção B', 'B'), ('Opção C', 'C')]:

tk.Radiobutton(janela, text=texto,

variable=opcao, value=val).pack(anchor='w')

# Para ler: opcao.get() → 'A', 'B' ou 'C'

```

2.6 Listbox

Exibe uma lista de itens selecionáveis.

```

listbox = tk.Listbox(janela, height=5,

selectmode=tk.SINGLE, # ou tk.MULTIPLE

font=('Arial', 11))

listbox.pack(padx=10, pady=5)


# Adicionar itens:

```

```

for fruta in ['Maçã', 'Banana', 'Laranja', 'Uva']:

    listbox.insert(tk.END, fruta)

# Ler seleção:

selecionados = listbox.curselection() # retorna índices

valor = listbox.get(selecionados[0]) # texto do item

```

2.7 Scale — Deslizador

```

scale = tk.Scale(janela,

from_=0, to=100,

orient=tk.HORIZONTAL, # ou VERTICAL

label='Volume',

length=200,

resolution=1,

command=lambda v: print(f'Valor: {v}'))

)

scale.set(50) # valor inicial

scale.pack()

# Ler: scale.get()

```

2.8 Combobox (ttk)

```

from tkinter import ttk

combo = ttk.Combobox(janela,

values=['Python', 'Java', 'C++', 'JavaScript'],

state='readonly',

```

```
font=('Arial', 11)

)

combo.set('Python') # valor inicial

combo.pack(pady=5)

# Ler: combo.get()
```

Gerenciadores de Layout

O Tkinter oferece três gerenciadores de layout para posicionar widgets: **pack**, **grid** e **place**. Escolha apenas um por container (frame ou janela) — misturar pack e grid no mesmo pai causa erros.

3.1 pack()

Posiciona widgets em sequência, empilhando-os horizontal ou verticalmente.

```
# Parâmetros principais:

widget.pack()

widget.pack(side=tk.TOP) # padrão: empilha de cima pra baixo

widget.pack(side=tk.LEFT) # coloca na esquerda

widget.pack(side=tk.RIGHT) # coloca na direita

widget.pack(side=tk.BOTTOM) # coloca em baixo


# Expansão e preenchimento:

widget.pack(fill=tk.X) # preenche largura

widget.pack(fill=tk.BOTH, expand=True) # preenche tudo


# Margem externa e interna:

widget.pack(padx=10, pady=5) # margem externa

widget.pack(ipadx=5, ipady=5) # margem interna


# Alinhamento num container horizontal:

widget.pack(side=tk.LEFT, anchor='n') # anchorar no topo
```

■ Bom uso do pack

Ideal para layouts simples, barras de ferramentas (side=LEFT) e empilhamento vertical de seções.

3.2 grid()

Posiciona widgets numa grade de linhas e colunas — ótimo para formulários.

```
# Sintaxe básica:

widget.grid(row=0, column=0)

widget.grid(row=0, column=1)


# Expandir para múltiplas colunas/linhas:

widget.grid(row=0, column=0, columnspan=2) # ocupa 2 colunas
widget.grid(row=0, column=0, rowspan=2) # ocupa 2 linhas


# Alinhamento interno na célula:

widget.grid(row=0, column=0, sticky='w') # alinha à esquerda
widget.grid(row=0, column=0, sticky='ew') # expande horizontal
widget.grid(row=0, column=0, sticky='nsew') # expande em todos lados


# Espaçamento:

widget.grid(row=0, column=0, padx=8, pady=4)


# Fazer coluna expandir com a janela:

janela.columnconfigure(1, weight=1)


# Exemplo: formulário de login

tk.Label(janela, text='Usuário:').grid(row=0, column=0, sticky='e')

tk.Entry(janela).grid(row=0, column=1, sticky='ew', padx=5)
```

```
tk.Label(janela, text='Senha:').grid(row=1, column=0, sticky='e')

tk.Entry(janela, show='*').grid(row=1, column=1, sticky='ew', padx=5)

tk.Button(janela, text='Entrar').grid(row=2, column=0, colspan=2, pady=8)
```

3.3 place()

Posicionamento absoluto (pixels) ou relativo (0.0 a 1.0). Use com moderação.

```
# Posição absoluta (pixels):

widget.place(x=100, y=50)

# Posição relativa (0.0 = esquerda/topo, 1.0 = direita/baixo):

widget.place(relx=0.5, rely=0.5, anchor='center') # centro da janela

# Tamanho relativo:

widget.place(relwidth=0.8, relheight=0.1, relx=0.1, rely=0.1)
```

■ Cuidado com place()

Posicionamento absoluto não se adapta quando a janela é redimensionada. Prefira pack() ou grid() na maioria dos casos.

3.4 Comparação e boas práticas

Gerenciador	Melhor para	Evitar quando
pack()	Layouts simples, barras	Formulários complexos
grid()	Formulários, tabelas	Layouts muito dinâmicos
place()	Sobreposição de widgets	Interfaces responsivas

Eventos e Callbacks

Eventos são ações do usuário (clique, tecla pressionada, movimento do mouse) ou do sistema (janela redimensionada). Callbacks são funções que executam em resposta a esses eventos. O Tkinter gerencia isso através do sistema de **binding**.

4.1 Binding de eventos

```
# Sintaxe: widget.bind('', funcao)

def ao_clicar(event):

    print(f'Clicado em x={event.x}, y={event.y}')

# Eventos de mouse:

janela.bind('', ao_clicar) # clique esquerdo

janela.bind('', ao_clicar) # clique direito

janela.bind('', func) # duplo clique

janela.bind('', func) # movimento do mouse

janela.bind('', func) # mouse entrou no widget

janela.bind('', func) # mouse saiu do widget

# Eventos de teclado:

janela.bind('', func) # qualquer tecla

janela.bind('', func) # Enter

janela.bind('', func) # Escape

janela.bind('', func) # Ctrl+S

janela.bind('', func) # F5

# Evento de janela:
```

```
janela.bind('', func) # redimensionamento

janela.protocol('WM_DELETE_WINDOW', confirmar_fechar)
```

4.2 Variáveis de controle

As variáveis de controle do Tkinter (**StringVar**, **IntVar**, **DoubleVar**, **BooleanVar**) são especiais pois permitem sincronizar valores entre widgets automaticamente e reagir a mudanças via **trace**.

```
# StringVar – texto

nome = tk.StringVar(value='João')

entry = tk.Entry(janela, textvariable=nome)

label = tk.Label(janela, textvariable=nome) # atualiza automático!


# IntVar – inteiro

idade = tk.IntVar(value=25)

spinbox = tk.Spinbox(janela, from_=0, to=120, textvariable=idade)


# BooleanVar – booleano

ativo = tk.BooleanVar(value=True)

check = tk.Checkbutton(janela, text='Ativo', variable=ativo)


# Trace: chama função quando o valor muda

def ao_mudar(nome_var, indice, operacao):

    print(f'Novo valor: {nome.get()}')

nome.trace_add('write', ao_mudar)
```

4.3 Lambda em callbacks

Quando você precisa passar parâmetros para um callback, use lambda:

```
# ERRADO – todas as cores serão 'azul' (closure problem)

for cor in ['vermelho', 'verde', 'azul']:

    tk.Button(janela, text=cor,

    command=lambda: mudar_cor(cor)).pack()


# CORRETO – captura o valor no momento da criação

for cor in ['vermelho', 'verde', 'azul']:

    tk.Button(janela, text=cor,

    command=lambda c=cor: mudar_cor(c)).pack()
```

■ Armadilha comum!

Lambda em loops sem parâmetro default captura a variável por referência, não por valor. Use 'lambda c=cor: ...' para fixar o valor atual.

Frames, Canvas e Menus

5.1 Frame — Contêiner de widgets

Frame é um contêiner invisível que agrupa widgets. Indispensável para organizar layouts complexos e isolar grupos de widgets com seu próprio gerenciador de layout.

```
frame_topo = tk.Frame(janela, bg='#e8f0fe', pady=8)

frame_topo.pack(fill=tk.X)

frame_corpo = tk.Frame(janela)

frame_corpo.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

# Widgets dentro do frame:

tk.Label(frame_topo, text='Cabeçalho', bg='#e8f0fe',
font=('Arial', 14, 'bold')).pack()

# LabelFrame — frame com título

grupo = tk.LabelFrame(janela, text='Informações',
font=('Arial', 10), padx=10, pady=8)

grupo.pack(padx=10, pady=10, fill=tk.X)

tk.Label(grupo, text='Nome:').grid(row=0, column=0)

tk.Entry(grupo).grid(row=0, column=1)
```

5.2 Canvas — Área de desenho

O Canvas permite desenhar formas, textos e imagens. Fundamental para gráficos, jogos e visualizações customizadas.

```
canvas = tk.Canvas(janela, width=400, height=300,
bg='white', highlightthickness=0)
```

```

canvas.pack()

# Desenhar formas:

canvas.create_rectangle(50, 50, 200, 150,
fill='#1a73e8', outline='#0d47a1', width=2)

canvas.create_oval(220, 50, 370, 150,
fill='#1D9E75', outline='#0F6E56')

canvas.create_line(50, 200, 370, 200,
fill='red', width=3, dash=(5, 3))

canvas.create_text(200, 250,
text='Texto no canvas',
font=('Arial', 12, 'bold'), fill='#202124')

# Mover um objeto:

id_rect = canvas.create_rectangle(10, 10, 60, 60, fill='blue')

canvas.move(id_rect, 100, 50) # desloca 100px direita, 50px baixo

# Deletar:

canvas.delete(id_rect)

canvas.delete('all') # limpa tudo

```

5.3 Menu

```

# Barra de menus

barra = tk.Menu(janela)

```

```

janela.config(menu=barra)

# Menu Arquivo

m_arquivo = tk.Menu(barra, tearoff=0)

barra.add_cascade(label='Arquivo', menu=m_arquivo)

m_arquivo.add_command(label='Novo', command=novo, accelerator='Ctrl+N')

m_arquivo.add_command(label='Abrir', command=abrir, accelerator='Ctrl+A')

m_arquivo.add_separator()

m_arquivo.add_command(label='Sair', command=janela.quit)

# Menu de contexto (clique direito)

menu_ctx = tk.Menu(janela, tearoff=0)

menu_ctx.add_command(label='Copiar', command=copiar)

menu_ctx.add_command(label='Colar', command=colar)

def mostrar_menu(event):

    menu_ctx.tk_popup(event.x_root, event.y_root)

janela.bind('', mostrar_menu)

```

5.4 Toplevel — Janelas secundárias

```

def abrir_janela():

    top = tk.Toplevel(janela)

    top.title('Configurações')

    top.geometry('300x200')

    top.grab_set() # torna modal (bloqueia janela principal)

```

```
tk.Label(top, text='Janela secundária').pack(pady=20)

tk.Button(top, text='Fechar',
command=top.destroy).pack()

tk.Button(janela, text='Abrir janela',
command=abrir_janela).pack()
```

Diálogos e MessageBox

O Tkinter traz diálogos prontos para mensagens, seleção de arquivos, entrada de dados e seleção de cores — sem necessidade de criar do zero.

6.1 messagebox

```
from tkinter import messagebox

# Informação

messagebox.showinfo('Sucesso', 'Arquivo salvo com sucesso!')

# Aviso

messagebox.showwarning('Atenção', 'Dados não salvos!')

# Erro

messagebox.showerror('Erro', 'Falha ao conectar ao banco!')

# Pergunta Sim/Não → retorna True ou False

resposta = messagebox.askyesno('Confirmar', 'Deseja sair?')

if resposta:

    janela.destroy()

# Pergunta OK/Cancelar → True ou False

ok = messagebox.askokcancel('Salvar', 'Salvar antes de fechar?')
```

6.2 filedialog

```
from tkinter import filedialog
```

```

# Abrir arquivo

caminho = filedialog.askopenfilename(

title='Selecionar arquivo',

initialdir='/',

filetypes=[('Arquivos de texto', '*.txt'),

('Todos', '.*')]

)

# Abrir múltiplos arquivos

arquivos = filedialog.askopenfilenames()

# Salvar arquivo

destino = filedialog.asksaveasfilename(

defaulttextextension='.txt',

filetypes=[('Texto', '*.txt')]

)

# Selecionar pasta

pasta = filedialog.askdirectory()

```

6.3 simpldialog e colorchooser

```

from tkinter import simpldialog, colorchooser

# Entrada simples de texto

nome = simpldialog.askstring('Nome', 'Digite seu nome:')

# Entrada de número inteiro

```

```
idade = simpledialog.askinteger('Idade', 'Quantos anos?',  
minvalue=0, maxvalue=120)  
  
# Entrada de número decimal  
  
preco = simpledialog.askfloat('Preço', 'Informe o preço:')  
  
# Seletor de cor → retorna ((R,G,B), '#hexcor')  
  
resultado = colorchooser.askcolor(color='#1a73e8',  
title='Escolha uma cor')  
  
if resultado[1]: # se o usuário não cancelou  
  
cor_hex = resultado[1] # ex: '#ff5733'
```

Estilização com ttk

O módulo **ttk** (Themed Tkinter) oferece widgets com aparência nativa de cada sistema operacional e suporte a temas customizáveis. Sempre que possível, prefira widgets ttk aos clássicos — eles têm visual mais moderno.

7.1 Widgets ttk principais

```
from tkinter import ttk

# Equivalentes com melhor aparência:

ttk.Button(janela, text='Botão').pack()

ttk.Label(janela, text='Rótulo').pack()

ttk.Entry(janela).pack()

ttk.Combobox(janela, values=['A', 'B', 'C']).pack()

ttk.Spinbox(janela, from_=0, to=10).pack()

ttk.Checkbutton(janela, text='Opção').pack()

ttk.Radiobutton(janela, text='Escolha').pack()

ttk.Scale(janela, from_=0, to=100).pack()

ttk.Progressbar(janela, length=200, mode='determinate').pack()

ttk.Separator(janela, orient='horizontal').pack(fill='x')
```

7.2 Style — Temas e estilos customizados

```
style = ttk.Style()

# Ver temas disponíveis:

print(style.theme_names())

# ('winnative', 'clam', 'alt', 'default', 'classic', ...)
```

```

# Aplicar tema:

style.theme_use('clam')

# Criar estilo customizado:

style.configure('Azul.TButton',

background='#1a73e8',

foreground='white',

font=('Arial', 11, 'bold'),

padding=(12, 6)

)

style.map('Azul.TButton',

background=[('active', '#1557b0'), ('pressed', '#0d47a1')]

)

# Usar o estilo:

ttk.Button(janela, text='Botão azul',

style='Azul.TButton').pack(pady=10)

```

7.3 Treeview — Tabelas e árvores

```

tree = ttk.Treeview(janela,

columns=('nome', 'idade', 'cidade'),

show='headings', # esconde a coluna de ícone

height=6

)

```

```

# Definir cabeçalhos:

tree.heading('nome', text='Nome')

tree.heading('idade', text='Idade')

tree.heading('cidade', text='Cidade')


# Larguras das colunas:

tree.column('nome', width=150)

tree.column('idade', width=60, anchor='center')

tree.column('cidade', width=120)


# Inserir dados:

dados = [('Ana', 28, 'São Paulo'), ('Carlos', 35, 'Rio de Janeiro')]

for linha in dados:

    tree.insert('', tk.END, values=linha)


tree.pack(padx=10, pady=5)


# Ler seleção:

def ao_selecionar(event):

    item = tree.selection()[0]

    valores = tree.item(item, 'values')

    print(valores)

tree.bind('<>', ao_selecionar)

```

7.4 Notebook — Abas

```
notebook = ttk.Notebook(janela)
```

```
notebook.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

# Criar abas:

aba1 = ttk.Frame(notebook)

aba2 = ttk.Frame(notebook)

aba3 = ttk.Frame(notebook)

notebook.add(aba1, text='Geral')

notebook.add(aba2, text='Avançado')

notebook.add(aba3, text='Sobre')

# Conteúdo nas abas:

ttk.Label(aba1, text='Conteúdo da aba 1').pack(pady=20)

# Mudar para uma aba específica:

notebook.select(1) # índice 0, 1, 2...
```

Introdução ao CustomTkinter

O **CustomTkinter** (CTk) é uma biblioteca moderna construída sobre o Tkinter que oferece widgets com design contemporâneo, suporte nativo a modo escuro, temas de cores personalizáveis e visual consistente em todos os sistemas operacionais.

8.1 Por que usar CustomTkinter?

Característica	Tkinter	CustomTkinter
Visual moderno	Não	Sim (cantos arredondados, cores)
Modo escuro	Manual	Automático
Temas de cores	Limitado	Completo
Scaling HiDPI	Manual	Automático
Incluso no Python	Sim	Instalar (pip)
Curva de aprendizado	Baixa	Muito baixa (mesma API)

8.2 Instalação

```
# Via pip:

pip install customtkinter

# Verificar instalação:

python -c "import customtkinter; print(customtkinter.__version__)"

# Atualizar:

pip install --upgrade customtkinter
```

8.3 Estrutura básica de um app CTK

```
import customtkinter as ctk
```

```

# Configurações globais (antes de criar qualquer widget)

ctk.set_appearance_mode('dark') # 'light', 'dark', 'system'

ctk.set_default_color_theme('blue') # 'blue', 'dark-blue', 'green'


# Janela principal

app = ctk.CTk()

app.title('App Moderno')

app.geometry('500x400')


# Widgets:

label = ctk.CTkLabel(app, text='Bem-vindo!',
font=ctk.CTkFont(size=20, weight='bold'))

label.pack(pady=20)


btn = ctk.CTkButton(app, text='Clique aqui',
command=lambda: print('Clicado!'))

btn.pack(pady=10)


app.mainloop()

```

■ Migração do Tkinter

A maioria dos widgets CTK tem a mesma API do Tkinter com prefixo 'CTk'. Mudar 'tk.Button' para 'ctk.CTkButton' já melhora o visual automaticamente!

Widgets do CustomTkinter

O CustomTkinter reimplementa os principais widgets com visual moderno. Abaixo, os mais utilizados com seus parâmetros principais.

9.1 CtkButton

```
btn = ctk.CtkButton(  
  
    master=app,  
  
    text='Salvar',  
  
    command=salvar,  
  
    width=140,  
  
    height=40,  
  
    corner_radius=8, # arredondamento  
  
    fg_color='#1a73e8', # cor de fundo  
  
    hover_color='#1557b0', # cor ao passar o mouse  
  
    text_color='white',  
  
    font=ctk.CtkFont(size=13, weight='bold'),  
  
    image=None, # CtkImage opcional  
  
    compound='left', # posição da imagem  
  
    state='normal' # ou 'disabled'  
  
)  
  
btn.pack(padx=20, pady=10)
```

9.2 CtkLabel e CtkEntry

```
# CtkLabel  
  
label = ctk.CtkLabel(  

```

```

master=app,

text='Nome: ',

font=ctk.CTkFont(size=14),

text_color='gray80',

anchor='w'

)

label.pack(anchor='w', padx=20)

# CTkEntry

entry = ctk.CTkEntry(

master=app,

placeholder_text='Digite seu nome...',

width=300,

height=38,

corner_radius=6,

border_width=2,

border_color='gray40'

)

entry.pack(padx=20, pady=5)

valor = entry.get()

```

9.3 CTkCheckBox e CTkRadioButton

```

# CTkCheckBox

var = ctk.StringVar(value='on')

check = ctk.CTkCheckBox(

```

```

master=app,

text='Receber notificações',

variable=var,

onvalue='on', offvalue='off',

checkbox_width=20, checkbox_height=20

)

check.pack(padx=20, pady=5)


# CTkRadioButton

opcao = ctk.StringVar(value='opcao1')

for txt, val in [('Opção 1', 'opcao1'), ('Opção 2', 'opcao2')]:

    ctk.CTkRadioButton(

        master=app, text=txt,

        variable=opcao, value=val

    ).pack(anchor='w', padx=30, pady=2)

```

9.4 CTkSlider e CTkProgressBar

```

# CTkSlider

def ao_mover(val):

    print(f'Valor: {val:.0f}')


slider = ctk.CTkSlider(

    master=app,

    from_=0, to=100,

    number_of_steps=100,

```

```

command=ao_mover,

orientation='horizontal',

width=300

)

slider.set(50)

slider.pack(padx=20, pady=10)


# CtkProgressBar

progresso = ctk.CTkProgressBar(app, width=300)

progresso.set(0.7) # 0.0 a 1.0

progresso.pack(padx=20, pady=10)


# Animação indeterminada:

progresso.configure(mode='indeterminate')

progresso.start()

```

9.5 CTkSwitch, CTkComboBox e CTkTextbox

```

# CTkSwitch

switch_var = ctk.StringVar(value='on')

switch = ctk.CTkSwitch(

master=app, text='Modo escuro',

variable=switch_var,

onvalue='on', offvalue='off',

command=lambda: ctk.set_appearance_mode(

'dark' if switch_var.get()=='on' else 'light')

```

```

)

switch.pack(padx=20, pady=10)

# CtkComboBox

combo = ctk.CtkComboBox(

master=app,

values=['Python', 'JavaScript', 'Rust'],

state='readonly',

width=200

)

combo.set('Python')

combo.pack(pady=5)

# CtkTextbox (multilinha)

textbox = ctk.CtkTextbox(

master=app,

width=350, height=120,

corner_radius=8,

wrap='word'

)

textbox.pack(padx=20, pady=10)

textbox.insert('0.0', 'Texto inicial...')

conteudo = textbox.get('0.0', 'end')

```

Layouts e Temas no CustomTkinter

10.1 CtkFrame e CtkScrollableFrame

```
# CtkFrame – contêiner com visual customizado

frame = ctk.CtkFrame(

    master=app,

    width=300, height=200,

    corner_radius=12,

    fg_color='gray20',

    border_width=1,

    border_color='gray40'

)

frame.pack(padx=20, pady=10)


# CtkScrollableFrame – frame com scroll automático

scroll_frame = ctk.CtkScrollableFrame(

    master=app,

    width=300, height=200,

    label_text='Lista de itens', # título opcional

    label_font=ctk.CtkFont(size=13, weight='bold')

)

scroll_frame.pack(padx=20, pady=10)


# Adicionar muitos itens sem preocupação com scroll:

for i in range(30):
```

```
ctk.CTkLabel(scroll_frame,  
  
text=f'Item {i+1}').pack(anchor='w', pady=2)
```

10.2 Modos claro e escuro

```
# Antes de criar a janela:  
  
ctk.set_appearance_mode('dark') # escuro  
  
ctk.set_appearance_mode('light') # claro  
  
ctk.set_appearance_mode('system') # segue o OS  
  
# Mudar durante a execução:  
  
def toggle_modos():  
  
    modo_atual = ctk.get_appearance_mode()  
  
    ctk.set_appearance_mode(  
  
        'dark' if modo_atual == 'Light' else 'light'  
  
    )  
  
ctk.CTkButton(app, text='Alternar tema',  
  
command=toggle_modos).pack(pady=10)
```

10.3 Temas de cores

```
# Temas padrão disponíveis:  
  
ctk.set_default_color_theme('blue') # azul (padrão)  
  
ctk.set_default_color_theme('dark-blue') # azul escuro  
  
ctk.set_default_color_theme('green') # verde  
  
# Tema customizado (arquivo JSON):
```

```
# Crie um arquivo 'meu_tema.json' e use:

ctk.set_default_color_theme('meu_tema.json')


# Estrutura mínima do arquivo JSON de tema:

# {

# "CTkButton": {

# "fg_color": ["#1a73e8", "#1557b0"],

# "hover_color": ["#1557b0", "#0d47a1"],

# "text_color": ["#ffffff", "#ffffff"]

# }

# }

# (primeiro valor = modo claro, segundo = modo escuro)
```

10.4 Scaling HiDPI

```
# Ajustar escala de toda a UI:

ctk.set_widget_scaling(1.5) # 150% – monitores HiDPI

ctk.set_window_scaling(1.0) # escala da janela


# Ou deixar o CustomTkinter detectar automaticamente:

# (padrão: já adapta ao DPI do monitor)
```

Projeto 1 — Calculadora (Tkinter)

Vamos construir uma calculadora funcional usando Tkinter puro. Este projeto consolida: grid layout, variáveis de controle, eventos e organização em classe.

```
import tkinter as tk

class Calculadora:

    def __init__(self):

        self.janela = tk.Tk()

        self.janela.title('Calculadora')

        self.janela.resizable(False, False)

        self.janela.configure(bg='#1c1c1c')

        self.expressao = ''

        self.var_display = tk.StringVar(value='0')

        self._criar_widgets()

    def _criar_widgets(self):

        # Display

        display = tk.Label(self.janela,

            textvariable=self.var_display,

            font=('Arial', 28, 'bold'),

            bg='#1c1c1c', fg='white',

            anchor='e', padx=15, pady=12)

        display.grid(row=0, column=0,

            columnspan=4, sticky='ew')
```

```

# Botões

botoes = [

('C', 1,0,'#555'), ('±', 1,1,'#555'),

('%', 1,2,'#555'), ('/', 1,3,'#ff9500'),

('7', 2,0,'#333'), ('8', 2,1,'#333'),

('9', 2,2,'#333'), ('*', 2,3,'#ff9500'),

('4', 3,0,'#333'), ('5', 3,1,'#333'),

('6', 3,2,'#333'), ('-', 3,3,'#ff9500'),

('1', 4,0,'#333'), ('2', 4,1,'#333'),

('3', 4,2,'#333'), ('+', 4,3,'#ff9500'),

('0', 5,0,'#333'), ('.', 5,2,'#333'),

('=', 5,3,'#ff9500'),

]

for texto, lin, col, cor in botoes:

cs = 2 if texto == '0' else 1

tk.Button(self.janela, text=texto,

command=lambda t=texto: self._click(t),

font=('Arial', 18), bg=cor,

fg='white', relief='flat',

width=4, height=2

).grid(row=lin, column=col,

columnspan=cs, padx=2, pady=2)

def _click(self, tecla):

```

```

if tecla == 'C':

    self.expressao = ''

    self.var_display.set('0')

elif tecla == '=':

    try:

        resultado = eval(self.expressao)

        self.var_display.set(resultado)

        self.expressao = str(resultado)

    except:

        self.var_display.set('Erro')

        self.expressao = ''

    else:

        self.expressao += tecla

        self.var_display.set(self.expressao)

def executar(self):

    self.janela.mainloop()

if __name__ == '__main__':

    app = Calculadora()

    app.executar()

```

Projeto 2 — App Moderno (CustomTkinter)

Neste projeto construímos um app com sidebar de navegação lateral e área de conteúdo principal — padrão comum em aplicações desktop modernas.

```
import customtkinter as ctk

ctk.set_appearance_mode('dark')

ctk.set_default_color_theme('blue')

class AppModerno(ctk.CTk):

    def __init__(self):

        super().__init__()

        self.title('App Moderno')

        self.geometry('800x520')

        self.minsize(600, 400)

        # Layout: sidebar (esquerda) + conteúdo (direita)

        self.grid_columnconfigure(1, weight=1)

        self.grid_rowconfigure(0, weight=1)

        self._criar_sidebar()

        self._criar_area_principal()

        self.mostrar_pagina('inicio')

    def _criar_sidebar(self):

        sb = ctk.CTkFrame(self, width=180, corner_radius=0)
```

```

sb.grid(row=0, column=0, sticky='nsew')

sb.grid_rowconfigure(5, weight=1)

ctk.CTkLabel(sb, text='Menu',

font=ctk.CTkFont(size=16, weight='bold')

).grid(row=0, column=0, padx=20, pady=(20,10))

self.btns_nav = {}

paginas = [('inicio', 'Início'), ('dados', 'Dados'),

('config', 'Configurações')]

for i, (key, nome) in enumerate(paginas):

btn = ctk.CTkButton(sb, text=nome,

corner_radius=6, height=36, anchor='w',

command=lambda k=key: self.mostrar_pagina(k))

btn.grid(row=i+1, column=0,

padx=10, pady=4, sticky='ew')

self.btns_nav[key] = btn

# Toggle modo escuro na sidebar

ctk.CTkSwitch(sb, text='Modo escuro',

command=self._toggle_tema

).grid(row=6, column=0, padx=20, pady=20, sticky='s')

def _criar_area_principal(self):

self.frame_conteudo = ctk.CTkFrame(self)

self.frame_conteudo.grid(row=0, column=1,

```

```

sticky='nsew', padx=10, pady=10)

self.frame_conteudo.grid_rowconfigure(1, weight=1)

self.frame_conteudo.grid_columnconfigure(0, weight=1)

self.lbl_titulo = ctk.CTkLabel(

self.frame_conteudo, text='',

font=ctk.CTkFont(size=20, weight='bold'))

self.lbl_titulo.grid(row=0, column=0,

padx=20, pady=(20,10), sticky='w')

self.area_scroll = ctk.CTkScrollableFrame(

self.frame_conteudo)

self.area_scroll.grid(row=1, column=0,

sticky='nsew', padx=10, pady=(0,10))

self.area_scroll.grid_columnconfigure(0, weight=1)

def mostrar_pagina(self, pagina):

titulos = {'inicio':'Início','dados':'Dados','config':'Configurações'}

self.lbl_titulo.configure(text=titulos[pagina])

for w in self.area_scroll.winfo_children():

w.destroy()

if pagina == 'inicio':

ctk.CTkLabel(self.area_scroll,

text='Bem-vindo ao app moderno!',

```

```

font=ctk.CTkFont(size=14)

).pack(anchor='w', pady=8)

ctk.CTkButton(self.area_scroll,

text='Ir para Dados',

command=lambda: self.mostrar_pagina('dados')

).pack(pady=10)

elif pagina == 'dados':

for i in range(10):

f = ctk.CTkFrame(self.area_scroll)

f.pack(fill='x', pady=4)

ctk.CTkLabel(f, text=f'Item {i+1}',

font=ctk.CTkFont(size=12)

).pack(side='left', padx=12, pady=8)

ctk.CTkButton(f, text='Ver',

width=60, height=28

).pack(side='right', padx=8)

elif pagina == 'config':

ctk.CTkLabel(self.area_scroll,

text='Aparência'

).pack(anchor='w', pady=(12,4))

ctk.CTkComboBox(self.area_scroll,

values=['Sistema', 'Claro', 'Escuro'],

command=lambda v: ctk.set_appearance_mode(

{'Claro': 'light', 'Escuro': 'dark'}.get(v, 'system'))

```

```
).pack(anchor='w', padx=10)

def _toggle_tema(self):

modo = ctk.get_appearance_mode()

ctk.set_appearance_mode(

'light' if modo == 'Dark' else 'dark')

if __name__ == '__main__':

AppModerno().mainloop()
```

Boas Práticas e Próximos Passos

13.1 Organize em classes

Nunca coloque toda a lógica em funções soltas. Herdar de **tk.Tk** ou **ctk.CTk** torna o código organizado, testável e reutilizável.

```
class MeuApp(tk.Tk): # ou ctk.CTk

    def __init__(self):

        super().__init__()

        self.title('App')

        self._configurar()

        self._criar_widgets()

        self._criar_menu()


    def _configurar(self): ...

    def _criar_widgets(self): ...

    def _criar_menu(self): ...

    # Callbacks separados por funcionalidade:

    def _salvar(self): ...

    def _abrir(self): ...


if __name__ == '__main__':

    app = MeuApp()

    app.mainloop()
```

13.2 Threading — operações longas sem travar a GUI

O Tkinter roda em uma única thread. Operações demoradas (rede, arquivo grande) travam a interface. Use threading para executá-las em background.

```

import threading

def tarefa_demorada():

    # Ex: download, processamento

    import time; time.sleep(3)

    # NUNCA atualize widgets direto aqui!

    # Use after() para voltar à thread da GUI:

    app.after(0, lambda: label.config(text='Concluído!'))

def iniciar():

    btn.config(state='disabled')

    t = threading.Thread(target=tarefa_demorada, daemon=True)

    t.start()

```

■ Regra de ouro

Jamais atualize widgets Tkinter/CTk de dentro de uma thread secundária. Use sempre `app.after(0, callback)` para delegar a atualização para a thread principal.

13.3 Empacotar com PyInstaller

```

# Instalar:

pip install pyinstaller

# Gerar executável simples:

pyinstaller --onefile --windowed meu_app.py

# Com ícone personalizado:

pyinstaller --onefile --windowed --icon=icone.ico meu_app.py

```

```
# Com CustomTkinter (incluir pasta de dados):

pyinstaller --onefile --windowed \

--add-data 'caminho/customtkinter;customtkinter' \

meu_app.py

# O executável fica em ./dist/
```

13.4 Recursos para continuar

- Documentação oficial Tkinter: docs.python.org/3/library/tkinter.html
- CustomTkinter GitHub: github.com/TomSchimansky/CustomTkinter
- Tkinter Tutorial (Real Python): realpython.com/python-gui-tkinter/
- Livro: 'Python GUI Programming with Tkinter' – Alan D. Moore
- Comunidade: r/learnpython, Stack Overflow (tag: tkinter)

■ Próximo nível

Depois de dominar Tkinter/CustomTkinter, explore PyQt6/PySide6 para apps corporativos mais complexos, ou Kivy para apps mobile/desktop multiplataforma.

Fim da apostila · Bons estudos!